im working on k means algo.

```
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid
distance.

    Q2(a): the two update functions together are worth 15 points.
    data has shape (n, d); centroids has shape (k, d).
    The result should be an integer array with shape (n,).
    Resolve equal-distance ties consistently and document your
rule.
    """
    # TODO: Compute distances and assign each point to a
centroid.
    assignment = np.array(data.shape[0],)
    for ptIdx in range(0, data.shape[0]):
        closestCentroidIdx = 0
        minDist = np.linalg.norm(centroids[0], data[ptIdx])
        for centeroidIdx in range(1, centroids.shape[0]):
            currDist = np.linalg.norm(centroids[centeroidIdx],
data[ptIdx])
            if(currDist < minDist):
                minDist = currDist
                closestCentroidIdx = centeroidIdx

            # if two centroids are at same distance from a point,
select a random one
            elif(currDist == minDist and rnd.random() > 0.5):
                closestCentroidIdx = centeroidIdx

        assignment[ptIdx] = closestCentroidIdx
        return assignment
```

Show more

good?

Close in structure, but there are several bugs that will stop it from working. Here they are in order of severity:

1. `np.linalg.norm(a, b)` **is wrong.** The second argument to `norm` is `ord`, not another vector. You need to norm the *difference*:

```python
np.linalg.norm(centroids[0] - data[ptIdx])
```

2. **`np.array(data.shape[0],)` doesn't create an empty array.** It makes a 0-d array holding the single value `n`. You want:

```python
assignment = np.zeros(data.shape[0], dtype=int)
```

3. **`return` is inside the for loop.** It's indented under `for ptIdx`, so the function returns after the first point. Dedent it to the function level.

4. **The random tie-break isn't uniform or reproducible.** With `rnd.random() > 0.5`, a 3-way tie doesn't give each centroid equal probability (the later ones are favored). More importantly, the docstring says to *"resolve ties consistently and document your rule."* Random choice is the opposite of consistent, since the same input can give different output between runs, which can also make k-means fail to converge cleanly. The simplest fix is to pick the lowest index, which is also what `np.argmin` does, and to say so in a comment.

Here's a corrected version that keeps your loop structure:

```python
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid distance.

    Ties: if a point is equidistant from several centroids, it is assigned
    to the one with the lowest index (deterministic).
    """
    assignment = np.zeros(data.shape[0], dtype=int)
    for ptIdx in range(data.shape[0]):
        closestCentroidIdx = 0
        minDist = np.linalg.norm(centroids[0] - data[ptIdx])
        for centroidIdx in range(1, centroids.shape[0]):
            currDist = np.linalg.norm(centroids[centroidIdx] - data[ptIdx])
            if currDist < minDist:  # strict <, so ties keep the lower index
                minDist = currDist
                closestCentroidIdx = centroidIdx
        assignment[ptIdx] = closestCentroidIdx
    return assignment
```

And if loops aren't required, a vectorized version is shorter and much faster:

python

```
def update_assignments(data, centroids):
    # (n, 1, d) - (1, k, d) -> (n, k, d) -> distances (n, k)
    dists = np.linalg.norm(data[:, None, :] - centroids[None, :, :], axis=2)
    return np.argmin(dists, axis=1)  # ties -> lowest index
```

Minor note: you can compare squared distances instead of norms to skip the square root, since it doesn't change which centroid is nearest.

Also watch for the second function (`update_centroids`): the usual gotcha there is an empty cluster, where taking the mean of zero points gives NaN. Decide how to handle that (keep the old centroid, or reinitialize to a random point).

```
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid
distance.

    Q2(a): the two update functions together are worth 15 points.
    data has shape (n, d); centroids has shape (k, d).
    The result should be an integer array with shape (n,).
    Resolve equal-distance ties consistently and document your
rule.
    """
    # TODO: Compute distances and assign each point to a
centroid.
    assignment = np.array(data.shape[0],dtype=int)
    for ptIdx in range(0, data.shape[0]):
        closestCentroidIdx = 0
        minDist = np.linalg.norm(centroids[0], data[ptIdx])
        for centeroidIdx in range(1, centroids.shape[0]):
            currDist = np.linalg.norm(centroids[centeroidIdx]-
data[ptIdx])
            if(currDist < minDist):
                minDist = currDist
                closestCentroidIdx = centeroidIdx

            # if two centroids are at same distance from a point,
select the one with lowest idx
            elif(currDist == minDist and closestCentroidIdx >
centeroidIdx):
                closestCentroidIdx = centeroidIdx

        assignment[ptIdx] = closestCentroidIdx
    return assignment
```

Show more

Better: the `return` is fixed, the tie rule is now deterministic, and one `norm` call is corrected. Three issues remain.

1. The first `norm` call is still wrong. You fixed it inside the inner loop but not on the line above it:

python

```
minDist = np.linalg.norm(centroids[0], data[ptIdx])   # still passes data as `ord`
```

Should be:

```python
minDist = np.linalg.norm(centroids[0] - data[ptIdx])
```

2. `np.array(data.shape[0], dtype=int)` is still a 0-d array. Adding `dtype` doesn't change the shape problem: it creates a single scalar holding `n`, so `assignment[ptIdx] = ...` will raise an `IndexError`. Use:

```python
assignment = np.zeros(data.shape[0], dtype=int)
```

3. The `elif` tie-break is dead code. Since you iterate `centeroidIdx` upward from 1, and `closestCentroidIdx` is always a smaller index than the current one, `closestCentroidIdx > centeroidIdx` can never be true. The `elif` never fires. That's harmless, because strict `<` already keeps the lowest index on ties, but it's clutter. Delete the `elif` and keep the comment, rewritten to explain why it works:

```python
# Strict '<' means on a tie the earlier (lower-index) centroid is kept.
```

Cleaned up:

```python
```

```
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid distance.

    Ties: a point equidistant from several centroids goes to the one
    with the lowest index (strict '<' keeps the earlier centroid).
    """
    assignment = np.zeros(data.shape[0], dtype=int)
    for ptIdx in range(data.shape[0]):
        closestCentroidIdx = 0
        minDist = np.linalg.norm(centroids[0] - data[ptIdx])
        for centroidIdx in range(1, centroids.shape[0]):
            currDist = np.linalg.norm(centroids[centroidIdx] - data[ptIdx])
            if currDist < minDist:
                minDist = currDist
                closestCentroidIdx = centroidIdx
        assignment[ptIdx] = closestCentroidIdx
    return assignment
```

Also, you may want to rename `centeroidIdx` to `centroidIdx` for consistency (typos in variable names are easy to miss when they're used in several places).

tell me about vectorized approach

The vectorized version replaces both Python loops with array operations that run in compiled C code, so it's typically 10-100x faster.

python

```
def update_assignments(data, centroids):
    diffs = data[:, None, :] - centroids[None, :, :]   # (n, k, d)
    dists = np.linalg.norm(diffs, axis=2)              # (n, k)
    return np.argmin(dists, axis=1)                    # (n,)
```

Step 1: Broadcasting

You want the difference between every point and every centroid. The trick is to reshape the arrays so NumPy pairs them up automatically:

- `data` has shape `(n, d)`; `data[:, None, :]` makes it `(n, 1, d)`
- `centroids` has shape `(k, d)`; `centroids[None, :, :]` makes it `(1, k, d)`

When shapes differ, NumPy stretches any dimension of size 1 to match the other. So `(n, 1, d) - (1, k, d)` becomes `(n, k, d)`, where `diffs[i, j]` is `data[i] - centroids[j]`. That is exactly what your inner loop computed one pair at a time.

Step 2: Distances

`np.linalg.norm(diffs, axis=2)` takes the norm along the last axis (the `d` coordinates), collapsing `(n, k, d)` to `(n, k)`. Now `dists[i, j]` is the distance from point `i` to centroid `j`.

Step 3: Argmin

`np.argmin(dists, axis=1)` finds, for each row (point), the column (centroid) with the smallest distance. It returns an integer array of shape `(n,)`, which is exactly what the docstring asks for. On ties, `argmin` returns the first (lowest) index, so your tie rule is built in. You should still state it in a comment, since the assignment says to document it.

Tradeoff: memory

The `(n, k, d)` intermediate array can be big. With n = 100,000 points, k = 10, d = 50, that's 50 million floats, about 400 MB. For typical homework datasets it's fine, but you can avoid it with the expansion

$\|x - c\|^2 = \|x\|^2 - 2x \cdot c + \|c\|^2$

python

```
def update_assignments(data, centroids):
    # Squared distances, shape (n, k); never builds the (n, k, d) array
    dists = (
        (data ** 2).sum(axis=1)[:, None]
        - 2 * data @ centroids.T
        + (centroids ** 2).sum(axis=1)[None, :]
    )
    return np.argmin(dists, axis=1)
```

This only needs `(n, k)` memory and uses a matrix multiply, which is very fast. Since you only need the *nearest* centroid, you can skip the square root (squaring preserves ordering).

One caveat: this version can have tiny floating-point errors, so two distances that are mathematically equal might differ in the last bits. That makes exact tie-breaking less reliable than in the broadcasting version. If your assignment is checked against exact ties, use the broadcasting version.

Which to use?

For an assignment, the first (broadcasting) version is the clearest and safest. Use the matrix-multiply version if you're clustering large data. If the assignment explicitly requires loops, stick with your loop version, but it's useful to keep the vectorized one around to check your answer: `np.array_equal(loop_result, vectorized_result)` should be `True`.

```
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid
distance.

    Q2(a): the two update functions together are worth 15 points.
    data has shape (n, d); centroids has shape (k, d).
    The result should be an integer array with shape (n,).
    Resolve equal-distance ties consistently and document your
rule.
    """
    # TODO: Compute distances and assign each point to a
centroid.
    assignment = np.zeros(data.shape[0], dtype=int)
    for ptIdx in range(0, data.shape[0]):
        closestCentroidIdx = 0
        minDist = np.linalg.norm(centroids[0] - data[ptIdx])
        for centeroidIdx in range(1, centroids.shape[0]):
            currDist = np.linalg.norm(centroids[centeroidIdx]-
data[ptIdx])
            if(currDist < minDist):
                minDist = currDist
                closestCentroidIdx = centeroidIdx

            # if two centroids are at same distance from a point,
select the one with lowest idx. Done implicitly

        assignment[ptIdx] = closestCentroidIdx
    return assignment
```

Show more

ok but this version will work fine?

Yes, this version is correct now. Both earlier bugs are fixed: the first `norm` call subtracts the vectors, and `assignment` is a proper length-`n` integer array. The `return` is at function level, and the tie rule works as your comment says. Strict `<` keeps the earlier centroid on ties, so equal distances go to the lowest index.

Two small things, neither of which affects correctness:

1. **Docstring/comment for the grader.** The assignment says to "document your rule," and a comment buried inside the loop is easy to miss. Put the rule in the docstring too, e.g. `Ties: a point equidistant from several centroids is assigned to the lowest-index centroid.`

2. **Typo.** `centeroidIdx` is consistently misspelled, so it runs fine, but renaming it to `centroidIdx` looks cleaner.

Make sure `numpy` is imported at the top of the file (`import numpy as np`), since the function uses `np` and the snippet doesn't show the import.

A quick sanity check you can run:

```python
data = np.array([[0., 0.], [10., 10.], [5., 5.]])
centroids = np.array([[0., 0.], [10., 10.]])
print(update_assignments(data, centroids))  # [0 1 0]
```

The point `[5, 5]` is exactly equidistant from both centroids, so it should go to index 0, which confirms the tie rule.

```
def update_centroids(data, centroids, assignments):
    """Return an updated (k, d) centroid array based on cluster
assignments.

    Document how you handle an empty cluster. One valid choice is
to retain
    its previous centroid. Avoid modifying the input centroid array
in place.
    """
Show more
    # TODO: Update the centroid of each nonempty cluster.
    # TODO: Handle empty clusters using your documented policy.
    newCentroids = np.zeros(centroids.shape, dtype=float)
    ptCount = np.zeros(assignments.shape, dtype=int)
    for i in range(0, assignments.shape[0]):
        newCentroids[assignments[i]] += data[i]
        ptCount[assignments[i]] += 1

    for i in range(0, newCentroids.shape[0]):
        if(ptCount > 0):
            newCentroids[i] /= ptCount[i]
        else:
            newCentroids[i] = centroids[i]
    return newCentroids
```

Good structure (accumulate sums, then divide), but there are two real bugs, both around `ptCount`.

1. `ptCount` has the wrong size. `np.zeros(assignments.shape)` makes it length `n` (one slot per point), but you need one count per *cluster*, so length `k`. With `n > k` this wastes space, but with `n < k` it would crash. And in the loop, `ptCount[assignments[i]]` indexes by cluster id, so it should be sized by `k`:

```python
ptCount = np.zeros(centroids.shape[0], dtype=int)
```

2. `if(ptCount > 0)` compares the whole array. This raises `ValueError: truth value of an array is ambiguous`. You meant the count for cluster `i`:

```python
if ptCount[i] > 0:
```

Fixed version:

```python
def update_centroids(data, centroids, assignments):
    """Return an updated (k, d) centroid array based on cluster assignments.

    Empty clusters: if no points are assigned to a cluster, it keeps its
    previous centroid. The input centroid array is not modified.
    """
    k = centroids.shape[0]
    newCentroids = np.zeros(centroids.shape, dtype=float)
    ptCount = np.zeros(k, dtype=int)

    for i in range(data.shape[0]):
        newCentroids[assignments[i]] += data[i]
        ptCount[assignments[i]] += 1

    for j in range(k):
        if ptCount[j] > 0:
            newCentroids[j] /= ptCount[j]
        else:
            newCentroids[j] = centroids[j]  # empty cluster: retain old centroid
    return newCentroids
```

Your original already avoids mutating the input `centroids` (you build `newCentroids` separately), which satisfies that requirement.

A vectorized alternative, if you want it:

python

```
def update_centroids(data, centroids, assignments):
    newCentroids = centroids.astype(float)  # astype returns a copy
    for j in range(centroids.shape[0]):
        members = data[assignments == j]
        if len(members) > 0:
            newCentroids[j] = members.mean(axis=0)
    return newCentroids
```

This one is shorter, and the empty-cluster policy falls out naturally because the copy already holds the old centroid.

Quick test:

python

```
data = np.array([[0., 0.], [2., 2.], [10., 10.]])
centroids = np.array([[1., 1.], [9., 9.], [50., 50.]])
assignments = np.array([0, 0, 1])
print(update_centroids(data, centroids, assignments))
# [[ 1.  1.]
#  [10. 10.]
#  [50. 50.]]   <- cluster 2 is empty, keeps old centroid
```

```
def update_centroids(data, centroids, assignments):
    """Return an updated (k, d) centroid array based on cluster
assignments.

    Document how you handle an empty cluster. One valid choice is
to retain
    its previous centroid. Avoid modifying the input centroid array
in place.
    """
    # TODO: Update the centroid of each nonempty cluster.
    # TODO: Handle empty clusters using your documented policy.
    newCentroids = np.zeros(centroids.shape, dtype=float)
    ptCount = np.zeros(centroids.shape[0], dtype=int)
    for i in range(0, assignments.shape[0]):
        newCentroids[assignments[i]] += data[i]
        ptCount[assignments[i]] += 1

    for i in range(0, newCentroids.shape[0]):
        if(ptCount[i] > 0):
            newCentroids[i] /= ptCount[i]
        else:
            newCentroids[i] = centroids[i]
    return newCentroids
```

Show more

Yes, this version is correct. Both fixes landed: `ptCount` is now sized by `k`, and the condition checks `ptCount[i]` instead of the whole array. The accumulate-then-divide logic is sound, and empty clusters fall back to the old centroid.

It also meets the "don't modify the input" requirement, since `newCentroids` is a separate array and `centroids` is only read from. One small detail worth knowing: `newCentroids[i] = centroids[i]` copies the values into your new array, so later changes to `newCentroids` won't leak back into the input.

Two things before you submit:

1. **Document the empty-cluster policy in the docstring.** The assignment explicitly says to document it, and right now the only hint is the code itself. Add something like:

python

```
Empty clusters: a cluster with no assigned points keeps its previous centroid.
```

2. **Integer data caveat.** You're safe here because `newCentroids` is created with `dtype=float`, so the division won't truncate even if `data` is an integer array. Good choice; keep that.

Quick test to confirm it behaves:

python

```
data = np.array([[0., 0.], [2., 2.], [10., 10.]])
centroids = np.array([[1., 1.], [9., 9.], [50., 50.]])
assignments = np.array([0, 0, 1])
print(update_centroids(data, centroids, assignments))
# [[ 1.  1.]
#  [10. 10.]
#  [50. 50.]]
```

Cluster 2 has no points, so it should keep `[50, 50]`, and the input `centroids` should be unchanged afterward.

If you want to tie both functions together, a minimal k-means loop is just:

python

```
for _ in range(max_iters):
    assignments = update_assignments(data, centroids)
    new_centroids = update_centroids(data, centroids, assignments)
    if np.allclose(new_centroids, centroids):
        break
    centroids = new_centroids
```

make a vectorized version

Building the bincount and sums arrays for the update ›

Here's a fully vectorized `update_centroids`, with no Python loops at all:

python

```
def update_centroids(data, centroids, assignments):
    """Return an updated (k, d) centroid array based on cluster assignments.

    Empty clusters: a cluster with no assigned points keeps its previous
    centroid. The input centroid array is not modified.
    """
    k = centroids.shape[0]

    counts = np.bincount(assignments, minlength=k)      # (k,) points per cluster
    sums = np.zeros(centroids.shape, dtype=float)       # (k, d)
    np.add.at(sums, assignments, data)                  # sums[assignments[i]] +=

    newCentroids = centroids.astype(float)              # copy, so the input is ur
    nonempty = counts > 0
    newCentroids[nonempty] = sums[nonempty] / counts[nonempty, None]
    return newCentroids
```

How it works

1. `np.bincount(assignments, minlength=k)` counts how many times each cluster id appears. `minlength=k` matters: without it, if the highest-numbered clusters are empty, the array would be shorter than `k`.
2. `np.add.at(sums, assignments, data)` does the accumulation your loop did (`newCentroids[assignments[i]] += data[i]`) in one call. You can't write `sums[assignments] += data` because NumPy's fancy-index `+=` doesn't accumulate repeated indices; it only applies one update per index. `np.add.at` handles repeats correctly.
3. `newCentroids = centroids.astype(float)` makes a copy holding the old centroids, so empty clusters keep their previous position automatically and the input isn't modified.
4. `nonempty = counts > 0` is a boolean mask. Only those rows get overwritten with `sums / counts`. The `counts[nonempty, None]` reshapes the counts to `(m, 1)` so the division broadcasts across the `d` coordinates.

Alternative: one-hot matrix multiply

This is often faster than `np.add.at`, which is known to be slowish:

```
python
```

```
def update_centroids(data, centroids, assignments):
    k = centroids.shape[0]
    onehot = (assignments[:, None] == np.arange(k)[None, :]).astype(float)  # (n,
    counts = onehot.sum(axis=0)                                             # (k,)
    sums = onehot.T @ data                                                  # (k,

    newCentroids = centroids.astype(float)
    nonempty = counts > 0
    newCentroids[nonempty] = sums[nonempty] / counts[nonempty, None]
    return newCentroids
```

The one-hot matrix has shape `(n, k)`, and `onehot.T @ data` sums the rows of `data` belonging to each cluster. It uses `O(n·k)` memory, which is fine unless `k` is large.

Test

python

```
data = np.array([[0., 0.], [2., 2.], [10., 10.]])
centroids = np.array([[1., 1.], [9., 9.], [50., 50.]])
assignments = np.array([0, 0, 1])
print(update_centroids(data, centroids, assignments))
# [[ 1.  1.]
#  [10. 10.]
#  [50. 50.]]
```

You can also check it against your loop version with `np.allclose(loop_result, vectorized_result)`. Use `allclose` rather than `array_equal` here, since summing in a different order can produce tiny floating-point differences.

Together with the vectorized `update_assignments` from earlier, the whole k-means iteration is loop-free apart from the outer iteration loop.

make a def plot_kmeans(data, centroids, assignments, name) that uses matplotlib to save picture of the graph with name filename to local path

python

```
import matplotlib.pyplot as plt

def plot_kmeans(data, centroids, assignments, name):
    """Save a scatter plot of the clustered data and centroids to `name`.

    Uses the first two features if data has more than 2 dimensions.
    `name` is the output filename (e.g. "kmeans.png"), saved relative to
    the current working directory.
    """
    k = centroids.shape[0]
    cmap = plt.get_cmap("tab10" if k <= 10 else "tab20")

    fig, ax = plt.subplots(figsize=(7, 6))

    # Points, colored by cluster
    ax.scatter(data[:, 0], data[:, 1], c=assignments, cmap=cmap,
               vmin=0, vmax=cmap.N - 1, s=20, alpha=0.7)

    # Centroids: large markers with a black outline, same color as their cluster
    ax.scatter(centroids[:, 0], centroids[:, 1],
               c=np.arange(k), cmap=cmap, vmin=0, vmax=cmap.N - 1,
               marker="X", s=250, edgecolors="black", linewidths=1.5,
               label="Centroids")

    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    ax.set_title(f"K-means clustering (k={k})")
    ax.legend()

    fig.tight_layout()
    fig.savefig(name, dpi=150)
    plt.close(fig)  # free memory; avoids stacking figures if called in a loop
```

Usage:

```python
plot_kmeans(data, centroids, assignments, "kmeans_result.png")
```

Notes:

- `vmin` / `vmax` are set so that cluster `j` gets the same color for both the points and its centroid. Without them, matplotlib rescales the color range to each call's data, and the

colors can mismatch.

- `plt.close(fig)` matters if you call this every iteration, to avoid piling up open figures and getting a memory warning.
- **Headless environments:** if you run on a server with no display and get a backend error, add `import matplotlib; matplotlib.use("Agg")` before importing `pyplot`.
- **Where it saves:** `name` is used as given, so `"out.png"` lands in the current working directory. To save elsewhere, pass a path like `"plots/out.png"` (the folder must exist, or create it with `os.makedirs("plots", exist_ok=True)`).
- **Iteration snapshots:** to save one image per iteration, use a different name each time, e.g. `plot_kmeans(data, centroids, assignments, f"iter_{it}.png")`.

numpy I have 2 arrays, x and y. y contains indeces into x. I want to create z where each element of z[i] is the element of x at index y[i]. How do vectorized?

Fancy indexing does exactly this:

```python
z = x[y]
```

`z[i]` is `x[y[i]]`, and `z` has the same shape as `y`. `y` must be an integer array (or list of ints).

```python
x = np.array([10, 20, 30, 40])
y = np.array([3, 0, 0, 2])
z = x[y]   # [40 10 10 30]
```

Indices can repeat, and they can be in any order. Negative indices work like normal Python (`-1` is the last element).

If `x` is 2-D, `x[y]` picks whole rows, so `z[i]` is the row `x[y[i]]` and `z` has shape `(len(y), x.shape[1])`. This is the same idea as `centroids[assignments]`, which gives each point its centroid. It's handy for k-means, e.g. computing the total squared error:

```python
inertia = np.sum((data - centroids[assignments]) ** 2)
```

For picking along a specific axis, or when `y` should select one element per row, `np.take(x, y, axis=...)` and `np.take_along_axis` are the alternatives. `np.take` also lets you

control out-of-range behavior with `mode="clip"` or `mode="wrap"`, whereas `x[y]` raises an `IndexError`.

```
def calc_total_clusters_varience(data, centroids, assignments):
  means = data[assignments]
  varience = (data - means)**2
```

Finding a bug in the cluster variance function.

Close, but there's a bug: `data[assignments]` indexes into `data`, not `centroids`. It "works" without an error only because both are arrays being indexed by integers, but it gives you the wrong points (and raises `IndexError` if any cluster id is >= n). You want each point's *centroid*:

python

```
def calc_total_clusters_variance(data, centroids, assignments):
    """Return the sum of squared distances from each point to its assigned centroi
    means = centroids[assignments]          # (n, d): centroid of each point's clu
    variance = (data - means) ** 2          # (n, d): squared per-coordinate diffe
    return variance.sum()                   # scalar
```

Your version also never summed or returned anything. `variance` is still an `(n, d)` array at that point, so the last line collapses it to a single number.

This quantity is usually called **inertia**, or within-cluster sum of squares (WCSS/SSE). It's the objective k-means minimizes, so it should never increase from one iteration to the next. That makes it a good correctness check for your two update functions.

Depending on what the assignment means by "variance," you may need a different normalization:

python

```
sse = variance.sum()                # total sum of squares (inertia)
mean_sq = sse / data.shape[0]       # average squared distance per point
```

If it wants per-cluster variances, you can compute them with `np.bincount`:

python

```
k = centroids.shape[0]
sq_dist = ((data - centroids[assignments]) ** 2).sum(axis=1)   # (n,) per-point
per_cluster_sse = np.bincount(assignments, weights=sq_dist, minlength=k)  # (k,)
```

Check the assignment text to see which of these it wants.

```
def calc_total_clusters_varience(data, centroids, assignments):
    means = centroids[assignments]
    varience = ((data - means)**2)/data.shape[0]
    return varience.sum()
```

This is correct for one particular definition: it returns the **mean squared distance per point**, i.e. inertia divided by `n`. Each squared difference is divided by `n` and then summed, which is the same as `sum / n`.

python

```
sse = ((data - means) ** 2).sum()
sse / data.shape[0]   # identical to your result
```

Whether it's the right answer depends on what the assignment calls "total clusters variance." Three common readings:

1. **Inertia / SSE** (sum of squared distances, no division). This is the standard k-means objective.
2. **Mean over points** (divide by `n`). This is what you have now.
3. **Sum of per-cluster variances**, where each cluster's squared distances are divided by *that cluster's* size, then added up. This gives a different number from yours, because it weights every cluster equally regardless of size.

If the assignment means #3, it looks like this:

python

```
def calc_total_clusters_variance(data, centroids, assignments):
    k = centroids.shape[0]
    sq_dist = ((data - centroids[assignments]) ** 2).sum(axis=1)       # (n,)
    sums = np.bincount(assignments, weights=sq_dist, minlength=k)      # (k,)
    counts = np.bincount(assignments, minlength=k)                     # (k,)
    per_cluster = np.divide(sums, counts, out=np.zeros(k), where=counts > 0)
    return per_cluster.sum()
```

The `where=counts > 0` guard avoids dividing by zero for empty clusters (they contribute 0).

Two small notes on your version:

- Dividing inside and then summing works, but `variance.sum() / data.shape[0]` is clearer and does one division instead of `n * d`.
- The name has a typo (`varience`), so if the grader or other code calls it by name, make sure the spelling matches what's expected (`variance`).

Check the assignment wording for which definition it wants. If it says something like "the k-means objective" or "total within-cluster sum of squares," drop the division. If it says "average" or "mean squared error," keep yours.

> def calc_total_clusters_variance(data, centroids, assignments): k = centroids.shape[0] sq_dist = ((data - centroids[assignments]) ** 2).sum(axis=1) # (n,) sums = np.bincount(assignments, weights=sq_dist, minlength=k) # (k,) counts = np.bincount(assignments, minlength=k) # (k,) per_cluster = np.divide(sums, counts, out=np.zeros(k), where=counts > 0) return per_cluster.sum()

Checking how the cluster sums are computed ›

Yes, that works. You pasted it on one line without a question, so I'm assuming you want to know whether it's correct. It is.

What it returns: the sum over clusters of each cluster's mean squared distance to its centroid. Empty clusters contribute 0.

Quick test:

```python
```

```
data = np.array([[0., 0.], [2., 2.], [10., 10.]])
centroids = np.array([[1., 1.], [10., 10.], [50., 50.]])
assignments = np.array([0, 0, 1])
print(calc_total_clusters_variance(data, centroids, assignments))  # 2.0
```

Cluster 0 has squared distances 2 and 2 (mean 2), cluster 1 has 0, and cluster 2 is empty (0), so the total is 2.0.

Things to watch:

1. **Not monotone.** Unlike inertia (the plain sum), this per-cluster-normalized quantity isn't guaranteed to decrease every k-means iteration. So don't use it as the convergence check or as a "should never go up" sanity test. For that, use `sq_dist.sum()`.
2. `assignments` **must be a non-negative integer array.** `np.bincount` raises an error on floats or negative values. If yours come from `np.argmin`, they're already fine.
3. **It's the population variance** (divides by cluster size, not size - 1), and it sums across all `d` dimensions, i.e. the trace of each cluster's covariance matrix. That's the usual convention for k-means, but check that it matches the assignment's definition.
4. **Style:** the one-liner is hard to read. Formatted normally, it's the version from my previous message.

If the assignment actually wanted a single overall number, the alternatives are `sq_dist.sum()` (inertia) or `sq_dist.mean()` (mean squared distance per point).

explain eahc line

Here's the function formatted properly, with each line explained.

python

```
def calc_total_clusters_variance(data, centroids, assignments):
    k = centroids.shape[0]
    sq_dist = ((data - centroids[assignments]) ** 2).sum(axis=1)
    sums = np.bincount(assignments, weights=sq_dist, minlength=k)
    counts = np.bincount(assignments, minlength=k)
    per_cluster = np.divide(sums, counts, out=np.zeros(k), where=counts > 0)
    return per_cluster.sum()
```

`k = centroids.shape[0]`
The number of clusters. `centroids` has shape `(k, d)`, so its first dimension is `k`. You need it so the later arrays have exactly one slot per cluster, even for empty clusters.

`sq_dist = ((data - centroids[assignments]) ** 2).sum(axis=1)`
This line does four things, inside out:

1. `centroids[assignments]` uses fancy indexing to build an `(n, d)` array where row `i` is the centroid of point `i`'s cluster.
2. `data - ...` subtracts it from the points, giving the `(n, d)` difference between each point and its own centroid.
3. `** 2` squares each coordinate difference.
4. `.sum(axis=1)` adds across the `d` coordinates, leaving an `(n,)` array where `sq_dist[i]` is the squared distance from point `i` to its centroid.

`sums = np.bincount(assignments, weights=sq_dist, minlength=k)`
Without `weights`, `bincount` counts how many times each cluster id appears. With `weights`, it instead adds up the weight of each occurrence. So `sums[j]` is the total of `sq_dist[i]` over all points `i` where `assignments[i] == j`: the total squared distance within cluster `j`. `minlength=k` guarantees length `k` even if the highest-numbered clusters are empty.

`counts = np.bincount(assignments, minlength=k)`
The same call without weights: `counts[j]` is the number of points in cluster `j`.

`per_cluster = np.divide(sums, counts, out=np.zeros(k), where=counts > 0)`
Element-wise `sums / counts`, giving each cluster's mean squared distance (its variance). Two arguments make it safe:

- `where=counts > 0` only performs the division where the cluster is nonempty, skipping the others, which would be 0/0.
- `out=np.zeros(k)` supplies the output array. Positions skipped by `where` are left untouched, so empty clusters stay at 0 instead of containing garbage or NaN.

`return per_cluster.sum()`
Adds the `k` per-cluster variances into one number.

Tiny example to trace it

```python
data = np.array([[0., 0.], [2., 2.], [10., 10.]])
centroids = np.array([[1., 1.], [10., 10.], [50., 50.]])
assignments = np.array([0, 0, 1])
```

- `k = 3`
- `sq_dist = [2, 2, 0]` (points 0 and 1 are each at squared distance 2 from `[1, 1]`; point 2 sits on its centroid)
- `sums = [4, 0, 0]` (cluster 0: 2 + 2; clusters 1 and 2: 0)
- `counts = [2, 1, 0]`
- `per_cluster = [2, 0, 0]` (4/2, 0/1, and the empty cluster skipped)

- result: `2.0`

```
def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b), 15 points: run all five k values and save cluster
plots.

    Use the four-component truth labels only for a reference
plot. K-means
    must cluster the features without access to those labels.
    """
    # TODO: Load toy data, create a seeded RNG, and initialize
centroids.
    # TODO: Run kmeans for every value in K_VALUES.
    # TODO: Plot each set of cluster assignments and its final
centroids.
    # TODO: Save plots under output_dir and discuss the effect of
k.
    data = make_toy_data()
    results = []
    k = 2
```

Show more

Checking assumptions about the helper functions' return values ›

You didn't include a question, so I'm assuming you want the function finished. I don't know the signatures of `make_toy_data`, `K_VALUES`, or your `kmeans`, so I made these assumptions:

- `make_toy_data()` returns `(features, truth_labels)`. If it returns only features, drop the labels and the reference plot.
- `K_VALUES` is a module-level list of five ints.
- There is no `kmeans` helper yet, so I wrote one using your two update functions. If you already have one, swap it in.

python

```
import os
import numpy as np
import matplotlib.pyplot as plt

def kmeans(data, k, rng, max_iterations=100):
    """Run k-means; return (centroids, assignments)."""
    # Initialize with k distinct random data points (no labels used)
    idx = rng.choice(data.shape[0], size=k, replace=False)
    centroids = data[idx].astype(float)

    for _ in range(max_iterations):
        assignments = update_assignments(data, centroids)
        new_centroids = update_centroids(data, centroids, assignments)
        if np.allclose(new_centroids, centroids):
            centroids = new_centroids
            break
        centroids = new_centroids

    assignments = update_assignments(data, centroids)  # match final centroids
    return centroids, assignments


def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b): run all K_VALUES and save cluster plots to output_dir.

    Truth labels are used only for the reference plot; k-means sees
    only the features.
    """
    os.makedirs(output_dir, exist_ok=True)
    data, truth = make_toy_data()
    rng = np.random.default_rng(seed)
    results = []

    # Reference plot: ground-truth components (not used by k-means)
    fig, ax = plt.subplots(figsize=(7, 6))
    ax.scatter(data[:, 0], data[:, 1], c=truth, cmap="tab10", s=20, alpha=0.7)
    ax.set_title("Ground-truth components (reference only)")
    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    fig.tight_layout()
    fig.savefig(os.path.join(output_dir, "truth.png"), dpi=150)
    plt.close(fig)
```

```
    for k in K_VALUES:
        centroids, assignments = kmeans(data, k, rng, max_iterations)
        inertia = calc_total_clusters_variance_sse(data, centroids, assignments)
        plot_kmeans(data, centroids, assignments,
                    os.path.join(output_dir, f"kmeans_k{k}.png"))
        results.append({"k": k, "centroids": centroids,
                        "assignments": assignments, "inertia": inertia})
        print(f"k={k}: inertia={inertia:.2f}")

    return results
```

I used `calc_total_clusters_variance_sse` as a placeholder. Inertia is the right quantity for comparing values of k, so use the plain sum version:

python

```
def calc_total_clusters_variance_sse(data, centroids, assignments):
    return ((data - centroids[assignments]) ** 2).sum()
```

or just inline `((data - centroids[assignments])**2).sum()`.

Discussion of k (the last TODO): you'll want to write this from your own results, but the expected pattern is:

- Inertia always decreases as k grows, and it reaches 0 when k = n. Lower inertia alone therefore doesn't mean a better clustering.
- Since the data has four true components, k < 4 should merge distinct groups, so the plots will show one centroid sitting between two real clusters and inertia dropping steeply as you add clusters.
- At k = 4 the clusters should match the truth reference plot, and past that the gains flatten (the "elbow"). Beyond 4, k-means splits real clusters into arbitrary pieces.
- Results depend on initialization. A bad seed can put two initial centroids in one component and leave another uncovered, so mention that you used a fixed seed (or try several seeds and keep the lowest inertia).

A chart of inertia vs. k is a good addition to support the discussion:

python

```
ks = [r["k"] for r in results]
plt.plot(ks, [r["inertia"] for r in results], marker="o")
plt.xlabel("k"); plt.ylabel("Inertia")
plt.savefig(os.path.join(output_dir, "inertia_vs_k.png"), dpi=150)
plt.close()
```

If `make_toy_data` returns something different, or if the assignment specifies an initialization method (e.g., k-means++ or the first k points), tell me and I'll adjust.

how to use rng to make random numbers in range

With a `np.random.default_rng(seed)` generator, the main methods are:

python

```
rng = np.random.default_rng(0)

# Random integers
rng.integers(0, 10)                # one int in [0, 10)  (high is exclusive)
rng.integers(0, 10, size=5)        # array of 5 ints
rng.integers(1, 7, size=(2, 3))    # 2x3 array, like dice rolls (1..6)
rng.integers(1, 6, endpoint=True)  # endpoint=True makes high inclusive: 1..6

# Random floats
rng.random()                       # one float in [0, 1)
rng.random(size=5)                 # array of 5 floats in [0, 1)
rng.uniform(2.0, 5.0, size=5)      # floats in [2.0, 5.0)

# Other distributions
rng.normal(loc=0, scale=1, size=5)  # Gaussian, mean 0, std 1
```

Scaling a float to any range (what `uniform` does for you):

python

```
low, high = 2.0, 5.0
x = low + (high - low) * rng.random(size=5)
```

Picking from existing data (the k-means case):

```python
idx = rng.choice(n, size=k, replace=False)   # k distinct indices from 0..n-1
centroids = data[idx]
```

`replace=False` guarantees no duplicates, which matters for initializing centroids, since two identical centroids would produce a permanently empty cluster.

Random points within the data's bounding box, another way to initialize centroids:

```python
mins, maxs = data.min(axis=0), data.max(axis=0)
centroids = rng.uniform(mins, maxs, size=(k, data.shape[1]))
```

`uniform` broadcasts `mins` and `maxs` across the columns, so each feature gets its own range.

Notes:

- The upper bound is exclusive for `integers` (unless `endpoint=True`) and for `uniform` / `random`.
- Pass the same `rng` object around rather than creating a new one each time. Creating `default_rng(0)` repeatedly gives you the same numbers every time. In `run_experiments`, the single `rng` is passed into `kmeans`, so each k gets a different but reproducible initialization.
- The older `np.random.randint` and `np.random.rand` use a hidden global state. Using `rng` keeps your results reproducible from the seed alone, which is what the assignment's `seed` parameter is for.

I have a list of tuples. I want to get the tuple with the largest third value from the list. how do

Use `max` with a `key` function:

```python
best = max(tuples, key=lambda t: t[2])
```

`key` tells `max` what to compare. Here it compares each tuple by its third element (index 2) and returns the whole tuple.

```python
```

```
tuples = [("a", 1, 5.2), ("b", 2, 9.1), ("c", 3, 7.0)]
max(tuples, key=lambda t: t[2])   # ("b", 2, 9.1)
```

You can use `operator.itemgetter` instead of a lambda, which is a bit faster and reads cleanly:

```python
from operator import itemgetter
best = max(tuples, key=itemgetter(2))
```

Notes:

- **Ties:** if several tuples share the largest third value, `max` returns the first one it encounters.
- **Empty list:** `max` raises `ValueError`. Pass a default to avoid that: `max(tuples, key=itemgetter(2), default=None)`.
- **Smallest instead:** use `min` the same way.
- **Index only:** if you want the position in the list rather than the tuple, use `max(range(len(tuples)), key=lambda i: tuples[i][2])`.
- **Top few:** for the top n, use `sorted(tuples, key=itemgetter(2), reverse=True)[:n]` or `heapq.nlargest(n, tuples, key=itemgetter(2))`.

For your k-means experiments, this is handy for picking the best run out of several seeds, e.g. if each tuple is `(centroids, assignments, inertia)`, you'd want `min` with `itemgetter(2)`, since lower inertia is better.

```
def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b), 15 points: run all five k values and save cluster
plots.

    Use the four-component truth labels only for a reference
plot. K-means
    must cluster the features without access to those labels.
    """
    # TODO: Load toy data, create a seeded RNG, and initialize
centroids.
    # TODO: Run kmeans for every value in K_VALUES.
    # TODO: Plot each set of cluster assignments and its final
centroids.
    # TODO: Save plots under output_dir and discuss the effect of
k.

    data = make_toy_data()
    rng = np.random.default_rng(seed)

    kmeans_times = 10
    for k in K_VALUES:
        results = []
        for i in range(0, kmeans_times):
            centroids = np.zeroes(k, data.shape[1])
            seen = set()
            # chose k random points to be initial centroids
            for i in range(0, k):
                x = rng.integers(0, data.shape[0])
                while x in seen:
                    x = rng.integers(0, data.shape[0])
                seen.add(x)
                centroids[i] = data[x]

            centroids, assignments = kmeans(data, centroids,
max_iterations)
            results.push(centroids, assignments,
calc_total_clusters_variance(data, centroids, assignments))

        best = max(results, key=lambda t: t[2])
        plot_kmeans(data, best[0], best[1], "graph k="+str(k))
```

Show more

Good direction: running 10 restarts per k and keeping the best is a solid idea. There are several bugs, though.

Bugs

1. `np.zeroes(k, data.shape[1])` has two problems: the function is `np.zeros`, and the shape must be one tuple: `np.zeros((k, data.shape[1]))`.
2. `results.push(...)` is JavaScript. Python lists use `append`, and it takes one argument, so wrap the values in a tuple: `results.append((centroids, assignments, score))`.
3. **Inner loop reuses `i`.** The initialization loop shadows the restart loop's `i`. It happens to work here since you don't use the outer `i`, but rename one of them.
4. **`max` should be `min`.** Lower variance/inertia is better.
5. **Wrong scoring function for comparing runs.** Your `calc_total_clusters_variance` normalizes per cluster, which isn't the k-means objective. Pick the best run by plain inertia (`((data - centroids[assignments])**2).sum()`).
6. `data = make_toy_data()` is a tuple `(features, truth)` if it returns labels, as I assumed earlier. If so, unpack it, and add the reference plot. If it returns only features, ignore this.
7. **The `kmeans(data, centroids, max_iterations)` signature** has to match your function. In my earlier version it took `(data, k, rng, max_iterations)` and did the initialization itself.
8. **Filename:** `"graph k=3"` has a space and no extension. Use `.png` and put it in `output_dir`, which you never use.

Simplification: the `seen` / `while` loop can be replaced by one call, which guarantees distinct indices:

```python
idx = rng.choice(data.shape[0], size=k, replace=False)
centroids = data[idx].astype(float)
```

Fixed version

```python
```

```
def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b): run all K_VALUES with restarts and save cluster plots."""
    os.makedirs(output_dir, exist_ok=True)
    data, truth = make_toy_data()          # drop `truth` if only features are ret
    rng = np.random.default_rng(seed)

    # Reference plot (truth labels are NOT used by k-means)
    fig, ax = plt.subplots(figsize=(7, 6))
    ax.scatter(data[:, 0], data[:, 1], c=truth, cmap="tab10", s=20, alpha=0.7)
    ax.set_title("Ground-truth components (reference only)")
    fig.tight_layout()
    fig.savefig(os.path.join(output_dir, "truth.png"), dpi=150)
    plt.close(fig)

    restarts = 10
    summary = []
    for k in K_VALUES:
        results = []
        for _ in range(restarts):
            # k distinct random data points as initial centroids
            idx = rng.choice(data.shape[0], size=k, replace=False)
            centroids = data[idx].astype(float)

            centroids, assignments = kmeans(data, centroids, max_iterations)
            inertia = ((data - centroids[assignments]) ** 2).sum()
            results.append((centroids, assignments, inertia))

        best = min(results, key=lambda t: t[2])   # lowest inertia wins
        plot_kmeans(data, best[0], best[1],
                    os.path.join(output_dir, f"kmeans_k{k}.png"))
        summary.append((k, best[2]))
        print(f"k={k}: best inertia={best[2]:.2f}")

    return summary
```

This assumes your `kmeans(data, centroids, max_iterations)` takes initial centroids and returns `(centroids, assignments)`. If it doesn't return final assignments consistent with the final centroids, call `update_assignments` once more after it returns.

Since the assignment says to *discuss the effect of k*, the returned `summary` (k vs. best inertia) is useful for an inertia-vs-k plot, as I showed earlier.

"""Question 2: K-means (30 points), adapted fro m Assignment 4, Questio n 2. Implement the two u pdate functions from scr atch, then complete th…

PASTED

Spotting a dtype regression in array assignment >

You pasted the full file without a question, so I reviewed it. It won't run yet. There's one regression and several bugs in `run_experiments`.

1. Regression in `update_assignments`: you changed the dtype to `float`.

```python
assignment = np.zeros(data.shape[0], dtype=float)   # should be dtype=int
```

Float indices break `centroids[assignments]`, `newCentroids[assignments[i]]` and `np.bincount`, all with errors. Change it back to `dtype=int`.

2. Bugs in `run_experiments`:

- `make_toy_data()` returns a tuple `(features, truth_labels)` from `make_blobs`, so `data.shape` fails. Unpack it: `data, truth = make_toy_data()`.
- `np.zeroes` should be `np.zeros`.
- `results.append(a, b, c)` takes one argument. Use `results.append((a, b, c))`.
- You select the best run with `calc_total_clusters_variance`, which normalizes per cluster and isn't the k-means objective. Choose by plain inertia (the sum of squared distances) instead.
- The plot goes to `"graph k=3"`, which has no extension and ignores `output_dir`.
- The assignment asks for a truth-label reference plot, and you don't have one yet.
- `output_dir` is never created.

3. Smaller points:

- Put the tie rule in the `update_assignments` docstring, since the assignment says to document it. The same goes for the empty-cluster policy in `update_centroids`.
- `get_initial_clusters` is provided for initialization, but your distinct-index approach is fine. It's even slightly safer, because duplicate starting centroids would leave one cluster permanently empty. Using the provided helper would be simpler:

```python
centroids = get_initial_clusters(k, data, rng)
```

Fixed `run_experiments`:

```python
```

```
def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b): run all K_VALUES with restarts, save cluster plots to output_dir."""
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    data, truth = make_toy_data()
    rng = np.random.default_rng(seed)

    # Reference plot only; k-means never sees `truth`
    fig, ax = plt.subplots(figsize=(7, 6))
    ax.scatter(data[:, 0], data[:, 1], c=truth, cmap="tab10", s=20, alpha=0.7)
    ax.set_title("Ground-truth components (reference only)")
    fig.tight_layout()
    fig.savefig(output_dir / "truth.png", dpi=150)
    plt.close(fig)

    restarts = 10
    summary = []
    for k in K_VALUES:
        results = []
        for _ in range(restarts):
            centroids = get_initial_clusters(k, data, rng)
            centroids, assignments = kmeans(data, centroids, max_iterations)
            inertia = ((data - centroids[assignments]) ** 2).sum()
            results.append((centroids, assignments, inertia))

        best = min(results, key=lambda t: t[2])     # lowest inertia wins
        plot_kmeans(data, best[0], best[1], str(output_dir / f"kmeans_k{k}.png"))
        summary.append((k, best[2]))
        print(f"k={k}: best inertia={best[2]:.2f}")

    # Optional: support the discussion with an inertia-vs-k plot
    ks, inertias = zip(*summary)
    plt.plot(ks, inertias, marker="o")
    plt.xlabel("k"); plt.ylabel("Inertia")
    plt.savefig(output_dir / "inertia_vs_k.png", dpi=150)
    plt.close()

    return summary
```

Two notes:

- `main()` calls `run_experiments(args.output_dir, ...)`, so a `Path` works as written.
- The last TODO, discussing the effect of k, is a written answer. Use your printed inertias and the plots: inertia always falls as k grows, k=4 should match the truth plot (four blobs), k<4 merges real clusters, and k>4 splits them. Mention that you used 10 restarts per k to reduce bad-initialization effects.

lets take this one step at a time. rewrite plot_kmeans to take output dir and rewrite the function to use inertia over varience

Here are both changes.

`plot_kmeans` with an output directory

python

```
def plot_kmeans(data, centroids, assignments, name, output_dir):
    """Save a scatter plot of clustered data and centroids to output_dir/name.

    Uses the first two features. `name` is a filename such as "kmeans_k3.png".
    The output directory is created if it doesn't exist.
    """
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    k = centroids.shape[0]
    cmap = plt.get_cmap("tab10" if k <= 10 else "tab20")

    fig, ax = plt.subplots(figsize=(7, 6))
    ax.scatter(data[:, 0], data[:, 1], c=assignments, cmap=cmap,
               vmin=0, vmax=cmap.N - 1, s=20, alpha=0.7)
    ax.scatter(centroids[:, 0], centroids[:, 1],
               c=np.arange(k), cmap=cmap, vmin=0, vmax=cmap.N - 1,
               marker="X", s=250, edgecolors="black", linewidths=1.5,
               label="Centroids")

    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")
    ax.set_title(f"K-means clustering (k={k})")
    ax.legend()

    fig.tight_layout()
    fig.savefig(output_dir / name, dpi=150)
    plt.close(fig)
```

Call it like this:

python

```
plot_kmeans(data, centroids, assignments, f"kmeans_k{k}.png", output_dir)
```

Inertia instead of variance

python

```
def calc_inertia(data, centroids, assignments):
    """Return the k-means objective: the sum of squared distances from each
    point to its assigned centroid (within-cluster sum of squares)."""
    return ((data - centroids[assignments]) ** 2).sum()
```

This replaces `calc_total_clusters_variance`, so you can delete the old function. Wherever you used it (in `run_experiments`, for picking the best restart), swap in `calc_inertia(data, centroids, assignments)`.

Next, when you're ready, is `run_experiments`.

```
def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b), 15 points: run all five k values and save cluster
plots.

    Use the four-component truth labels only for a reference
plot. K-means
    must cluster the features without access to those labels.
    """
    # TODO: Load toy data, create a seeded RNG, and initialize
centroids.
    # TODO: Run kmeans for every value in K_VALUES.
    # TODO: Plot each set of cluster assignments and its final
centroids.
    # TODO: Save plots under output_dir and discuss the effect of
k.

    data, truth = make_toy_data()
    rng = np.random.default_rng(seed)

    kmeans_times = 10
    for k in K_VALUES:
        results = []
        for _ in range(0, kmeans_times):
            centroids = np.zeros((k, data.shape[1]))
            seen = set()
            # chose k random points to be initial centroids
            for i in range(0, k):
                x = rng.integers(0, data.shape[0])
                while x in seen:
                    x = rng.integers(0, data.shape[0])
                seen.add(x)
                centroids[i] = data[x]

            centroids, assignments = kmeans(data, centroids,
max_iterations)
            results.append((centroids, assignments,
calc_inertia(data, centroids, assignments)))

        best = min(results, key=lambda t: t[2])
        plot_kmeans(data, centroids, assignments,
f"kmeans_k{k}.png", output_dir)
```

Show more

Nearly there. There's one real bug, plus a few missing pieces.

Bug: you plot the last restart, not the best one. You compute `best` and then pass `centroids, assignments`, which are just whatever the final loop iteration left behind. Use `best`:

```python
plot_kmeans(data, best[0], best[1], f"kmeans_k{k}.png", output_dir)
```

Missing pieces

1. **Truth reference plot.** The assignment asks for one, and `truth` is unpacked but never used. `plot_kmeans` colors by `assignments`, so you can reuse it by passing `truth` as the assignments. The "centroids" would then be the mean of each true component:

```python
true_centers = np.array([data[truth == j].mean(axis=0) for j in range(4)])
plot_kmeans(data, true_centers, truth, "truth.png", output_dir)
```

The title will say "K-means clustering (k=4)", which is slightly misleading for a truth plot. If that bothers you, add an optional `title` parameter to `plot_kmeans`.

2. **Record inertia per k** so you can write the discussion. Printing it is enough:

```python
print(f"k={k}: best inertia={best[2]:.2f}")
```

3. **Docstring and discussion.** The last TODO asks you to discuss the effect of k. That's a written answer, so a short comment block or a separate write-up works. I can help with it once you have the numbers.

Optional cleanup: your initialization loop works, but `get_initial_clusters` (provided in the file) does the same thing in one line. It samples with replacement, so duplicates are possible. Your `seen` set avoids that, which is a fine reason to keep yours. If you want the short version without duplicates:

```python
```

```
idx = rng.choice(data.shape[0], size=k, replace=False)
centroids = data[idx].astype(float)
```

Updated function

python

```python
def run_experiments(output_dir, seed=0, max_iterations=100):
    """Q2(b): run all K_VALUES with restarts and save cluster plots."""
    data, truth = make_toy_data()
    rng = np.random.default_rng(seed)

    # Reference plot only; k-means never sees `truth`
    true_centers = np.array([data[truth == j].mean(axis=0) for j in range(4)])
    plot_kmeans(data, true_centers, truth, "truth.png", output_dir)

    kmeans_times = 10
    for k in K_VALUES:
        results = []
        for _ in range(kmeans_times):
            idx = rng.choice(data.shape[0], size=k, replace=False)
            centroids = data[idx].astype(float)

            centroids, assignments = kmeans(data, centroids, max_iterations)
            inertia = calc_inertia(data, centroids, assignments)
            results.append((centroids, assignments, inertia))

        best = min(results, key=lambda t: t[2])   # lowest inertia wins
        plot_kmeans(data, best[0], best[1], f"kmeans_k{k}.png", output_dir)
        print(f"k={k}: best inertia={best[2]:.2f}")
```

One more thing to check in your file: `update_assignments` still had `dtype=float` in the version you pasted earlier. If you haven't changed it back to `dtype=int`, the whole thing will crash on indexing.

I traced through this version and it should run end to end now. Both earlier problems are fixed: `dtype=int` is back in `update_assignments`, and you plot `best` instead of the last restart. No question came with the paste, so here is what's left to polish.

1. Required by the assignment

- **Document the rules in the docstrings.** The assignment says "document your rule" twice, and right now the rules live only in inline comments. Add one line to each docstring:
  - `update_assignments`: `Ties: a point equidistant from several centroids goes to the lowest-index centroid (strict '<' keeps the earlier one).`
  - `update_centroids`: `Empty clusters: a cluster with no assigned points keeps its previous centroid.`
- **The discussion of k.** This is the last TODO, and it's a written answer. Add `print(f"k={k}: best inertia={best[2]:.2f}")` after the plot call so you have numbers to cite, then write it from your results.

2. Optional

- **Truth plot title.** It currently says "K-means clustering (k=4)". If that bothers you, add an optional `title=None` parameter to `plot_kmeans` and use `ax.set_title(title or f"K-means clustering (k={k})")`.
- **Initialization.** Your `seen` / `while` loop works. It's equivalent to:

```python
idx = rng.choice(data.shape[0], size=k, replace=False)
centroids = data[idx].astype(float)
```

- **Typo.** `centeroidIdx` is still misspelled, but consistently, so it's harmless.
- **Speed.** `update_assignments` is loop-based, and with 4000 points, 5 values of k, 10 restarts and 100 iterations each, that's 50 runs of 100 iterations. It will take a while, possibly minutes. If it's too slow, swap in the vectorized version I showed earlier, or break out of `kmeans` early when the centroids stop changing:

```python
new_centroids = update_centroids(data, centroids, assignments)
if np.allclose(new_centroids, centroids):
    centroids = new_centroids
    break
centroids = new_centroids
```

Since `kmeans` is described as the "provided" loop, check whether you're allowed to modify it. If not, vectorizing `update_assignments` is the safer speedup.

When you've run it and have the inertia numbers, send them over and I can help you write the discussion.

```
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid
distance.

    Q2(a): the two update functions together are worth 15 points.
    data has shape (n, d); centroids has shape (k, d).
    The result should be an integer array with shape (n,).
    Resolve equal-distance ties consistently and document your
rule.
    """
    # TODO: Compute distances and assign each point to a centroid.
    assignment = np.zeros(data.shape[0], dtype=int)
    for ptIdx in range(0, data.shape[0]):
        closestCentroidIdx = 0
        minDist = np.linalg.norm(centroids[0] - data[ptIdx])
        for centeroidIdx in range(1, centroids.shape[0]):
            currDist = np.linalg.norm(centroids[centeroidIdx]-
data[ptIdx])
            if(currDist < minDist):
                minDist = currDist
                closestCentroidIdx = centeroidIdx

            # if two centroids are at same distance from a point,
select the one with lowest idx. Done implicitly

        assignment[ptIdx] = closestCentroidIdx
    return assignment




def update_centroids(data, centroids, assignments):
    """Return an updated (k, d) centroid array based on cluster
assignments.

    Document how you handle an empty cluster. One valid choice is
to retain
    its previous centroid. Avoid modifying the input centroid array
in place.
```

```
    """
    # TODO: Update the centroid of each nonempty cluster.
    # TODO: Handle empty clusters using your documented policy.
    newCentroids = np.zeros(centroids.shape, dtype=float)
    ptCount = np.zeros(centroids.shape[0], dtype=int)
    for i in range(0, assignments.shape[0]):
        newCentroids[assignments[i]] += data[i]
        ptCount[assignments[i]] += 1

    for i in range(0, newCentroids.shape[0]):
        if(ptCount[i] > 0):
            newCentroids[i] /= ptCount[i]
        else: #empty clusters preserve their original mean
            newCentroids[i] = centroids[i]
    return newCentroids
```

runs but very slow. is it because not vectorized?

Show more

Yes, mostly. The main culprit is `update_assignments`, and `update_centroids` is a smaller contributor.

Why it's slow

- `update_assignments` runs a Python-level double loop: `n × k` iterations, each calling `np.linalg.norm` on a tiny vector. `norm` has a lot of per-call overhead (roughly a few microseconds), which dwarfs the actual arithmetic on 2 numbers. With n = 4000 and k up to 10, that's up to 40,000 `norm` calls per iteration.
- It runs inside `kmeans` for 100 iterations, 10 restarts, and 5 values of k. That's 5,000 iterations in total, so roughly 100 million `norm` calls across the whole experiment.
- `update_centroids` has a 4000-iteration Python loop per call, which is cheaper but still adds up over 5,000 calls.

Vectorizing moves the loops into compiled code, so you should see a 50-100x speedup or more.

Vectorized drop-in replacements

python

```
def update_assignments(data, centroids):
    """Return one cluster index per row, using nearest-centroid distance.

    Ties: a point equidistant from several centroids is assigned to the
    lowest-index centroid (np.argmin returns the first minimum).
    """
    # (n, 1, d) - (1, k, d) -> (n, k, d) -> distances (n, k)
    dists = np.linalg.norm(data[:, None, :] - centroids[None, :, :], axis=2)
    return np.argmin(dists, axis=1)


def update_centroids(data, centroids, assignments):
    """Return an updated (k, d) centroid array based on cluster assignments.

    Empty clusters: a cluster with no assigned points keeps its previous
    centroid. The input centroid array is not modified.
    """
    k = centroids.shape[0]
    counts = np.bincount(assignments, minlength=k)
    sums = np.zeros(centroids.shape, dtype=float)
    np.add.at(sums, assignments, data)

    newCentroids = centroids.astype(float)      # copy; empty clusters keep old va
    nonempty = counts > 0
    newCentroids[nonempty] = sums[nonempty] / counts[nonempty, None]
    return newCentroids
```

At this data size ($4000 \times 10 \times 2$) the `(n, k, d)` intermediate is tiny, so the broadcasting version is fine.

`np.add.at` is itself somewhat slow, but it's still far faster than a Python loop. If you want the fastest option, replace the `sums` lines with `sums = np.stack([np.bincount(assignments, weights=data[:, j], minlength=k) for j in range(data.shape[1])], axis=1)`, which loops only over the 2 features.

Verify it matches your loop version

Before switching, confirm identical results on random data:

```
python
```

```
rng = np.random.default_rng(1)
data = rng.normal(size=(500, 2))
cents = data[:4].copy()
a_loop = update_assignments_loop(data, cents)      # your original, renamed
a_vec = update_assignments(data, cents)
print(np.array_equal(a_loop, a_vec))               # True
print(np.allclose(update_centroids_loop(data, cents, a_loop),
                  update_centroids(data, cents, a_vec)))   # True
```

If the assignment requires loops

If "from scratch" means no vectorization, keep your loops and cut the work instead:

1. Break out of `kmeans` early when the centroids stop changing (K-means on this easy 4-blob data usually converges in 10-20 iterations, not 100).
2. Reduce `kmeans_times` from 10 to, say, 3 while testing.
3. Replace `np.linalg.norm(a - b)` in your loop with a plain squared-distance computation (`diff = a - b; diff @ diff`), which is much cheaper per call and gives the same nearest centroid.

The vectorized NumPy code still counts as "from scratch" in most courses, since you're not calling a library k-means, but check the wording if you're unsure.

what should I write for discuss k

Estimating expected inertia for the blob data ›

Write it from your own printed inertias, but here's a draft you can adapt. Fill in the bracketed numbers from your output and check each claim against your plots.

Discussion: effect of k

K-means minimizes the within-cluster sum of squared distances (inertia), so inertia decreases monotonically as k increases. Extra centroids can only reduce, never increase, the distance from points to their nearest centroid. Lower inertia alone therefore doesn't show that a larger k is a better clustering.

The data has four true components, and the plots show the following:

- **k = 2 and k = 3 (underfitting):** Distinct true clusters are merged, and a centroid sits between groups rather than at a group's center. Inertia is high [your values], and adding a cluster gives a large drop [k=2 → k=3 drop].

- **k = 4 (matches the structure):** Each centroid lands on one true component, and the clusters match the truth reference plot. Inertia drops sharply here [value]. As a sanity check, the data was generated with std 0.6 in 2D, so the expected inertia is about $4000 \times 2 \times 0.6^2 \approx 2{,}900$. Your number should be close.
- **k = 6 and k = 10 (overfitting):** Inertia keeps falling, but only slightly [values]. The extra centroids split real, compact clusters into arbitrary pieces, and the boundaries between those pieces cut through dense regions with no real separation. The lower inertia reflects fitting noise, not new structure.

Plotting inertia against k shows an **elbow** at k = 4: a steep decline up to four clusters and a flat one afterward. This is a standard heuristic for choosing k when the true number is unknown.

Initialization: K-means converges to a local minimum that depends on the starting centroids. For example, two initial centroids in the same blob can leave another blob covered by only one centroid, or merged with a neighbor. I used 10 random restarts per k and kept the run with the lowest inertia to reduce this, and fixed the seed for reproducibility.

Tips

- The elbow claim is much stronger with an `inertia_vs_k.png` plot. Collect `(k, best[2])` in `run_experiments` and plot it (the snippet is in my earlier message).
- Compare k=4 against `truth.png`. If your k=4 result doesn't match the truth plot, mention it and explain it with initialization. Even the best of 10 restarts can occasionally land in a bad minimum.
- Replace my expected behavior with what you actually see. If k=3 happens to separate the clusters well because two blobs sit close together, say so.
- Put this in a comment block at the bottom of `run_experiments` or in a separate write-up, depending on what the assignment asks for.

If you paste your printed inertias, I can tighten the numbers into the text.

"""

K means algorithm is always trying to reduce cluster varience, so the higher the k is, the lower the varience is for each cluster and so is the total varience

having a K which is too low groups unrelated clusters into one, while a k which is too high splits clusters into many

k = 2, 3 is underfitting.
k = 4 is perfect match. Each clear cluster has its own centroid
k > 4 is overfitting

increasing k from 2 to 3 and 4 results in very large varience drops, but increasing beyond results in very low varience drops, signifying that we k=4 is

```
optimal
    """
```

This captures the right ideas, and the structure (underfitting, match, overfitting, diminishing returns) is what the assignment is looking for. It's thin on evidence and has a few wording problems, so here's what I'd fix.

Corrections

1. **"Variance" vs. inertia.** You switched the code to inertia, so the text should say inertia (or "within-cluster sum of squared distances") to match your outputs. Also, per-cluster variance doesn't strictly fall as k rises, because adding a cluster can leave some clusters with higher spread. Total inertia is what's guaranteed not to increase.
2. **"Always trying to reduce" is imprecise.** Better: each iteration of k-means never increases inertia, so it converges to a local minimum, and the minimum achievable inertia can only fall as k grows.
3. **"We k=4"** is a typo for "we know k=4" or "k=4 is optimal."
4. **"Perfect match"** is a strong claim. Say it matches the truth plot, and only if your `kmeans_k4.png` really does.
5. **"Increasing from 2 to 3 and 4 results in very large drops"** should be backed by your numbers. Without them it's an unsupported claim.
6. **No mention of initialization or restarts.** You ran 10 restarts per k, so say so.

Revised version

python

```
"""
Effect of k:

K-means minimizes inertia (the sum of squared distances from each point to
its assigned centroid). Adding a centroid can only lower or keep the minimum
achievable inertia, so inertia falls as k grows. Lower inertia alone therefore
does not mean a better clustering.

Results (best of 10 random restarts per k; inertia from my run):
  k=2:  [value]
  k=3:  [value]
  k=4:  [value]
  k=6:  [value]
  k=10: [value]

- k = 2, 3 (underfitting): distinct true clusters are merged, and some
  centroids sit between groups instead of at a cluster center. Going from
  k=2 to 3 to 4 gives large inertia drops.
- k = 4 (matches the structure): each centroid sits on one of the four true
  components, matching the truth plot. This is where the inertia curve bends.
- k = 6, 10 (overfitting): inertia keeps falling, but only slightly. The
  extra centroids split real clusters into arbitrary pieces, so the lower
  inertia reflects fitting noise rather than new structure.

The sharp decline up to k=4 followed by a flat tail (the "elbow") indicates
k=4 as the best choice, which agrees with the four true components.

K-means only finds a local minimum that depends on initialization, so I used
10 restarts per k and kept the lowest-inertia run, with a fixed seed for
reproducibility.
"""
```

Fill in the bracketed inertias from your run. To print them, add this after the plot call in `run_experiments`:

python

```
print(f"k={k}: best inertia={best[2]:.2f}")
```

If you also save an `inertia_vs_k.png`, reference it in the text as evidence for the elbow. If you paste your numbers, I can check that the claims match them.

Work in `noisy_classification_template.py`. Helpers for generating clean data and training/evaluating the SVM are provided.
Generate independent training and test sets. Each set contains:

- 1,000 positive examples from a Gaussian with mean (1, 1) and covariance I.
- 1,000 negative examples from a Gaussian with mean (-1, -1) and covariance 3I.

Keep the test labels clean throughout.
(a) Noisy-label baseline — 20 points. Complete `corrupt_training_labels()` and the baseline portion of `run_comparison()`. Independently flip each original positive training label with probability 0.35, and each original negative training label with probability 0.20. Apply the probabilities to the original labels so that an example cannot be flipped twice. Train an RBF SVM with C = 1 and gamma = 0.01.
Keep the generated training and test sets fixed, repeat label corruption and training 20 times, and report the mean accuracy on the clean test set. Use a seed for reproducibility.
(b) K-NN label repair — 20 points. Complete `repair_labels_knn()` and the comparison portion of `run_comparison()`. For each training point:

1. Find its five nearest other training points using Euclidean distance.
2. Replace its label with the majority of those neighbors' noisy labels.
3. Compute every repaired label from the same original noisy-label array, rather than modifying labels sequentially.

A neighbor-search library is allowed. Five binary votes cannot tie; use a consistent rule if distances tie when selecting neighbors. Train a fresh SVM with the same parameters on the repaired labels. For each trial, use the same features and corrupted labels as the baseline.
Save a CSV with one row per trial and columns `trial`, `baseline_accuracy`, and `knn_accuracy`. Report both mean accuracies and their difference. Briefly explain whether K-NN repair helped and why it might help or hurt. Improvement is not guaranteed. No parameter sweep or additional clustering-based repair experiment is required.
The repair method may use only training features and noisy training labels. Do not pass it clean training labels or test labels. K-NN is a preprocessing step here; the final classifier is an SVM.

read this

Show more

Outlining the steps and pitfalls for the template task ›

I've read the task, but the template file didn't come through (only your message is visible). Paste `noisy_classification_template.py` and I'll fit the code to its exact helper names and signatures. In the meantime, here's the plan and a draft of the two functions.

Things to get right

1. **Flip against the original labels.** Draw one uniform number per example and compare it to a per-example threshold (0.35 for original positives, 0.20 for original negatives). One draw per example means nothing can be flipped twice.
2. **Fixed data, fresh noise.** Generate train and test once, then redo corruption and training in each of the 20 trials. Use one seeded `rng` throughout.
3. **Paired comparison.** In each trial, the baseline SVM and the K-NN SVM must use the same features and the same corrupted labels. Corrupt once, then train on both.
4. **Test labels stay clean**, and the repair function only sees `X_train` and the noisy labels.
5. **Exclude the point itself from its neighbors.** With sklearn, `NearestNeighbors.kneighbors()` called with no query argument automatically excludes each point from its own neighbor list.
6. **Repair from the original noisy array.** Compute all new labels from `y_noisy` in one shot, then return a new array.

Draft code (assumes labels are +1/-1; adjust if the template uses 0/1)

python

```
import numpy as np
from sklearn.neighbors import NearestNeighbors

def corrupt_training_labels(y_true, rng, p_flip_pos=0.35, p_flip_neg=0.20):
    """Flip each original label independently; probabilities depend on the
    ORIGINAL class, so no example can be flipped twice."""
    p = np.where(y_true == 1, p_flip_pos, p_flip_neg)
    flip = rng.random(y_true.shape[0]) < p
    return np.where(flip, -y_true, y_true)


def repair_labels_knn(X_train, y_noisy, k=5):
    """Replace each label with the majority noisy label of its k nearest
    OTHER training points. All repairs use the same original y_noisy.

    Distance ties when choosing neighbors: sklearn's brute-force search
    breaks them by lowest index (state this in your write-up only if you
    force algorithm='brute'; otherwise say the tie rule is the library's
    consistent, deterministic ordering).
    """
    nn = NearestNeighbors(n_neighbors=k, algorithm="brute").fit(X_train)
    idx = nn.kneighbors(return_distance=False)   # (n, k), excludes self
    votes = y_noisy[idx].sum(axis=1)             # sum of +/-1; odd k => never 0
    return np.where(votes > 0, 1, -1)
```

And the core of `run_comparison`:

```python
rng = np.random.default_rng(seed)
# ... generate X_train, y_train, X_test, y_test once ...
rows = []
for t in range(20):
    y_noisy = corrupt_training_labels(y_train, rng)
    base = SVC(kernel="rbf", C=1, gamma=0.01).fit(X_train, y_noisy)
    y_rep = repair_labels_knn(X_train, y_noisy)
    knn = SVC(kernel="rbf", C=1, gamma=0.01).fit(X_train, y_rep)
    rows.append((t, base.score(X_test, y_test), knn.score(X_test, y_test)))
# write CSV with columns trial, baseline_accuracy, knn_accuracy
# report means and mean(knn) - mean(baseline)
```

Write-up ideas (check against your results)

- **Why it might help:** flips are random, so in regions dominated by one class most of a point's 5 neighbors carry the correct label, and the majority vote undoes isolated flips.
- **Why it might hurt:** the classes overlap heavily (means about 2.8 apart, with the negative class much wider), so near the boundary the true neighbors are mixed. Voting there pushes labels toward the local majority and can erase real minority-class points. The noise is also asymmetric (35% vs 20%), so noisy labels are biased toward the negative class, and the vote can reinforce that bias. Repair can also leave correlated errors, since neighboring points share neighbors.

Send the template and I'll fill in the exact pieces.

```
def corrupt_training_labels(clean_labels, rng):
    """Independently flip +1 with probability .35 and -1 with
probability .20.

    Select flip probabilities from the ORIGINAL labels. Do not
allow a label
    flipped in one direction to be flipped again by the other rule.
    Return a new array, leaving clean_labels unchanged.
    """

    corrupt_labels = np.array(clean_labels)
    for label in corrupt_labels:
        if label > 0 and rng.random() < POSITIVE_FLIP_PROBABILITY:
            label *= -1
        elif label < 0 and rng.random() <
NEGATIVE_FLIP_PROBABILITY:
                    label *= -1
    return corrupt_labels
```

Show more

Checking whether the fix by index avoids double flips ›

There's one bug, and it means **no labels are ever flipped**: `for label in corrupt_labels` gives you a copy of each value, so `label *= -1` changes only that loop variable and never touches the array. The function returns an unchanged copy, so your "noisy" baseline would just be the clean baseline.

Iterate by index and write back into the array:

python

```
def corrupt_training_labels(clean_labels, rng):
    """Independently flip +1 with probability .35 and -1 with probability .20.

    Flip probabilities are selected from the ORIGINAL labels, and each label
    gets exactly one random draw, so nothing can be flipped twice.
    Returns a new array; clean_labels is unchanged.
    """
    corrupt_labels = np.array(clean_labels)          # copy
    for i in range(len(clean_labels)):
        if clean_labels[i] > 0 and rng.random() < POSITIVE_FLIP_PROBABILITY:
            corrupt_labels[i] = -clean_labels[i]
        elif clean_labels[i] < 0 and rng.random() < NEGATIVE_FLIP_PROBABILITY:
            corrupt_labels[i] = -clean_labels[i]
    return corrupt_labels
```

I check `clean_labels[i]` rather than `corrupt_labels[i]` so the decision always comes from the original label. Since each label is either positive or negative, only one branch ever draws a random number, which is what you want.

The vectorized version is shorter and does the same thing:

python

```
def corrupt_training_labels(clean_labels, rng):
    p_flip = np.where(clean_labels > 0,
                      POSITIVE_FLIP_PROBABILITY,
                      NEGATIVE_FLIP_PROBABILITY)     # per-example threshold
    flip = rng.random(len(clean_labels)) < p_flip    # one draw per example
    return np.where(flip, -clean_labels, clean_labels)
```

Both consume one random number per example in order, so they produce identical output for the same seed.

A quick sanity check after fixing it:

python

```
rng = np.random.default_rng(0)
y = np.concatenate([np.ones(1000), -np.ones(1000)])
yn = corrupt_training_labels(y, rng)
print((yn[:1000] != 1).mean())   # about 0.35
print((yn[1000:] != -1).mean())  # about 0.20
print(y[:3])                     # unchanged
```

If you'd like, paste `repair_labels_knn` and `run_comparison` next and I'll review those the same way.

does sklearn have a good knn libary/

Yes. sklearn has two relevant options, and for your task `NearestNeighbors` is the better fit.

`sklearn.neighbors.NearestNeighbors` finds neighbors and returns their indices, which is what you need for the repair step:

```python
from sklearn.neighbors import NearestNeighbors

def repair_labels_knn(X_train, y_noisy, k=5):
    nn = NearestNeighbors(n_neighbors=k).fit(X_train)
    idx = nn.kneighbors(return_distance=False)   # (n, k); excludes each point its
    votes = y_noisy[idx].sum(axis=1)             # labels are +/-1, k odd => never
    return np.where(votes > 0, 1, -1)
```

Calling `kneighbors()` with no query argument is the key detail: sklearn then automatically leaves each training point out of its own neighbor list, which satisfies "five nearest *other* training points." If you instead passed `X_train` as the query, each point's nearest neighbor would be itself at distance 0, and you'd have to ask for k+1 and drop the first column.

`sklearn.neighbors.KNeighborsClassifier` is a classifier. It would predict labels, but it can't easily exclude each point from its own vote when predicting on the training set, so it's the wrong tool for leave-one-out repair. You'd use it if K-NN were the final model, but here the final classifier is the SVM.

Details worth knowing

- **Algorithm choice:** the default `algorithm="auto"` picks a KD-tree or ball tree for low-dimensional data like yours. With 2000 points in 2D, speed is a non-issue either way. Passing `algorithm="brute"` gives a simple, predictable tie rule (lower index wins when distances tie), which makes it easy to document the "consistent rule" the assignment asks for.
- **Distance:** the default metric is Euclidean (`minkowski` with `p=2`), as the task requires.
- **Ties in distance:** with continuous Gaussian data, exact ties essentially never happen, so this is mostly a documentation point.
- **Computed once:** the repair uses `y_noisy[idx]` on the original array, so every repaired label comes from the same noisy labels, never from partially updated ones.

Alternatives if you ever want them: `scipy.spatial.cKDTree` (use `query(X, k=k+1)` and drop the first column) or `faiss` for huge datasets. Neither is needed here.

> def repair_labels_knn(X_train, y_noisy, k=5): nn = NearestNeighbors(n_neighbors=k).fit(X_train) idx = nn.kneighbors(return_distance=False) # (n, k); excludes each point itself votes = y_noisy[idx].sum(axis=1) # labels are +/-1, k odd => never 0 return np.where(votes > 0, 1, -1)
>
> explain each line

Here it is formatted, then each line explained.

```python
def repair_labels_knn(X_train, y_noisy, k=5):
    nn = NearestNeighbors(n_neighbors=k).fit(X_train)
    idx = nn.kneighbors(return_distance=False)
    votes = y_noisy[idx].sum(axis=1)
    return np.where(votes > 0, 1, -1)
```

`def repair_labels_knn(X_train, y_noisy, k=5):`
Takes the training features `(n, d)`, the noisy labels `(n,)` (values +1/-1), and the number of neighbors, defaulting to 5. It receives only features and noisy labels, so it can't see clean labels, as the assignment requires.

`nn = NearestNeighbors(n_neighbors=k).fit(X_train)`
Creates a neighbor-search object that returns the `k` closest points by default (Euclidean distance), then `.fit(X_train)` stores the training points and builds an internal search structure (a KD-tree or similar, chosen automatically). Nothing is searched yet; this just prepares the index.

`idx = nn.kneighbors(return_distance=False)`
This is the actual search. Because no query points are passed, sklearn uses the training points themselves as queries and automatically excludes each point from its own neighbor list. The result `idx` has shape `(n, k)`: row `i` holds the indices of the 5 nearest *other* points to point `i`. `return_distance=False` skips returning the distances, since you only need which points are neighbors.

`votes = y_noisy[idx].sum(axis=1)`
Two steps:

1. `y_noisy[idx]` is fancy indexing. It replaces every index in `idx` with that point's noisy label, producing an `(n, k)` array where row `i` is the five neighbors' noisy labels.

2. `.sum(axis=1)` adds across each row. Since labels are +1 or -1, the sum is the vote tally: positive means more +1 votes, negative means more -1 votes.

For example, neighbor labels `[+1, -1, -1, +1, -1]` sum to `-1`, so the majority is -1.

Because `idx` is computed once and `y_noisy` is never modified, every repaired label comes from the same original noisy array, as the assignment requires.

`return np.where(votes > 0, 1, -1)`
Converts each tally into a label: where `votes > 0` is true, use `1`, otherwise `-1`. With 5 votes of ±1 the sum is odd (±1, ±3, ±5), so it is never 0 and there's no tie to resolve. The result is a new array of length `n`, and `y_noisy` is left unchanged.

One caveat: this assumes labels are exactly +1 and -1. If your template uses 0/1 labels, the sum trick breaks. In that case use `votes = y_noisy[idx].sum(axis=1)` and `return (votes >= 3).astype(int)` (3 of 5 votes for class 1 is a majority).

Tiny trace

python

```
y_noisy = np.array([1, -1, -1, 1, -1, 1])
# suppose point 0's neighbors are idx[0] = [1, 2, 3, 4, 5]
y_noisy[[1, 2, 3, 4, 5]]        # [-1, -1, 1, -1, 1]
# sum = -1  ->  repaired label for point 0 is -1
```

"""Question 3: noisy-label SVM and K-NN repair (40 points). Complete the TODOs. Keep test labels clean and use them only for evaluation. """ import...

PASTED

any bugs? finish run comparison function without chanign anything else

Checking the label-repair function's tie handling ›
I found no bugs in the rest of the file. Two notes, neither requiring a change:

- The default output path is `results/noise_comparison.csv`, and that folder may not exist, so `run_comparison` creates it.
- `repair_labels_knn` uses the default `algorithm="auto"`. Exact distance ties essentially never occur with continuous Gaussian data, so for the write-up you can say ties are broken by the library's deterministic ordering.

Here is `run_comparison`, with everything else left untouched. I put the `csv` import inside the function so nothing at the top of the file changes.

python

```
def run_comparison(repeats, seed, neighbors, output_path):
    """Q3(a-b), 20 points each: compare the baseline with K-NN label repair.

    Suggested adaptation: generate independent clean training/test sets once,
    then repeat corruption and training (default: 20 noise trials). In each
    trial, use identical data and corrupted labels for both approaches.
    Do not give the repair method the clean training or test labels.
    """
    import csv

    rng = np.random.default_rng(seed)

    # Independent clean splits, generated once and kept fixed
    train_features, train_labels = generate_clean_split(rng)
    test_features, test_labels = generate_clean_split(rng)

    rows = []
    for trial in range(repeats):
        # Fresh corruption each trial (test labels stay clean)
        noisy_labels = corrupt_training_labels(train_labels, rng)

        # Baseline: SVM trained directly on the noisy labels
        baseline_acc = train_and_evaluate(train_features, noisy_labels,
                                          test_features, test_labels)

        # K-NN repair uses only training features and the same noisy labels
        repaired_labels = repair_labels_knn(train_features, noisy_labels, neighbor
        knn_acc = train_and_evaluate(train_features, repaired_labels,
                                     test_features, test_labels)

        rows.append((trial, baseline_acc, knn_acc))

    # Save per-trial results
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    with open(output_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["trial", "baseline_accuracy", "knn_accuracy"])
        writer.writerows(rows)

    # Report means and the change from baseline
    baseline_mean = float(np.mean([r[1] for r in rows]))
```

```
    knn_mean = float(np.mean([r[2] for r in rows]))
    print(f"Baseline mean accuracy: {baseline_mean:.4f}")
    print(f"K-NN repair mean accuracy: {knn_mean:.4f}")
    print(f"Difference (knn - baseline): {knn_mean - baseline_mean:+.4f}")

    return baseline_mean, knn_mean
```

What it does, step by step:

1. **Seeding:** one `rng` seeded from `seed` is used for the data and the noise, so the whole run is reproducible.
2. **Fixed data:** train and test are generated once, from different draws of the same `rng`, so they're independent.
3. **Paired comparison:** each trial corrupts the labels once and uses that same `noisy_labels` array for both the baseline and the repair, as the assignment requires.
4. **No leakage:** `repair_labels_knn` is passed only `train_features` and `noisy_labels`. The clean labels go only into the corruption step, and the test labels only into evaluation.
5. **Output:** the CSV has columns `trial`, `baseline_accuracy` and `knn_accuracy`, one row per trial, and the printed means and difference are what you report.

When you have the numbers, paste them and I can help check the write-up on whether K-NN helped. With heavily overlapping classes and asymmetric noise, don't assume it will. If the difference is small, compare it to the trial-to-trial spread before claiming an improvement.

